The three-residual attention block, trained with an annealed diagonal bias: structure, initialization, pretraining and attention

Experiment log, branch `attention-experiments`

2026-10-19

Summary

The *three_residual* block puts a residual connection on each of the three matrices of an attention head: the attention matrix A_h, the value matrix W_{vh} and the output matrix W_O. Trained from scratch on OpenWebText as written, it collapses: within 100 steps the loss stops at the unigram level (7.54 vs. 7.591) and never moves. The collapse is already visible at initialization, where the residual stream is nearly rank 1 by layer 11 (distance from rank 1: 0.148; standard GPT-2: 0.713).

Adding a bias $b = 8$ to every head's self logit at initialization, then lowering it linearly to 0 over the first 1000 steps, fixes this. From step 1000 on the model is exactly three_residual again, and it keeps training:

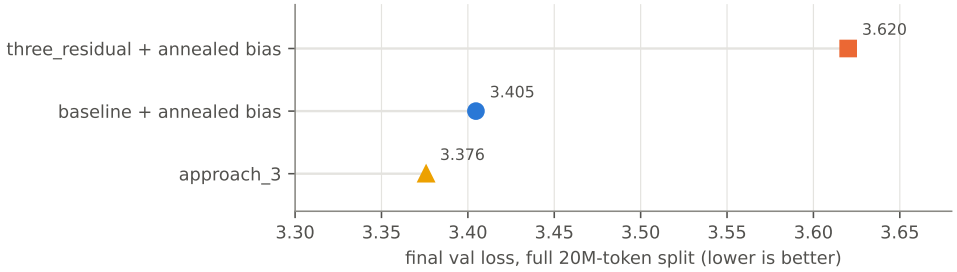


Figure 1: Final validation loss after 2.5B OpenWebText tokens, on the full 20M-token validation split. The *baseline + annealed bias* control is standard GPT-2 trained with the same bias schedule.

- **The collapse was a bad starting point, not something the block cannot learn.** With the annealed bias, three_residual reaches 3.620, compared with 3.405 for the control. The gap, 0.215 nats, comes from the block, since both runs used the same bias schedule.
- **The trained three_residual mostly attends to the token itself.** The bias is 0 for the last 3768 steps, yet 94 of 144 heads put more than half their weight on the token itself (control: 14). With no small-scale skip path around the attention term, the model keeps each token's state by making A_h close to the identity.

1 Setup

Data	OpenWebText, 24 of its 80 parquet shards spread evenly; GPT-2 BPE train 2.69B tokens; validation 20M tokens (the end of the prepared stream)
Budget	2.5B tokens (Chinchilla, ≈ 20 per parameter): 4768 steps $\times$ 524,288 tokens
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1 (matrices only), gradient clip 1.0
Learning rate	linear warmup over 250 steps to 6×10^{-4}, cosine decay to 6×10^{-5}
Batch	16 micro-batches of 32×1024 tokens, accumulated; bf16 autocast; one H100 GPU per run
Seed and order	seed 1337 and the same sequential data order in every run
Evaluation	4M validation tokens every 250 steps; the full 20M tokens at the end

All losses in this report are on the same OpenWebText validation split, so they are comparable with each other. They are *not* comparable with the FineWeb-Edu numbers of the 2026-10-15/16 reports. The unigram model (token frequencies only, no context) scores 7.591 on this split.

2 Structure

2.1 The block

For head h (12 heads, $d = D/H = 64$), let X_h be the head's d-dimensional slice of the raw residual stream X (before LN_1), and A_h its causal attention matrix, computed from $\mathrm{LN}_1(X)$:

$$A_h = \mathrm{softmax}\Big(\tfrac{1}{\sqrt{d}}\, Q_h K_h^\top + b\, I + \mathrm{mask}\Big) \qquad Q_h, K_h \text{ from } \mathrm{LN}_1(X) \tag{1}$$

$$Z_h = A_h X_h + X_h \qquad \text{residual on } A_h \tag{2}$$

$$Y = \mathrm{Concat}_h\big[\, Z_h W_{vh} + Z_h \,\big] \qquad \text{residual on } W_{vh} \tag{3}$$

$$X' = Y W_O + b_O + Y \qquad \text{residual on } W_O \tag{4}$$

$$X^{(\ell+1)} = X' + \mathrm{MLP}(\mathrm{LN}_2(X')) \qquad \text{standard MLP sublayer} \tag{5}$$

The bias b exists only during pretraining (Section 3); with $b = 0$ these equations are exactly the block in `src/model.py` (`three_residual=True`). W_{vh} is a separate 64×64 matrix per head, so the value weights are block-diagonal ($12 \times 64 \times 64 = 49$k parameters, against 590k for GPT-2's full W_V), and LN_1 only feeds Q and K. The model has 117,978,624 parameters (GPT-2: 124,475,904).

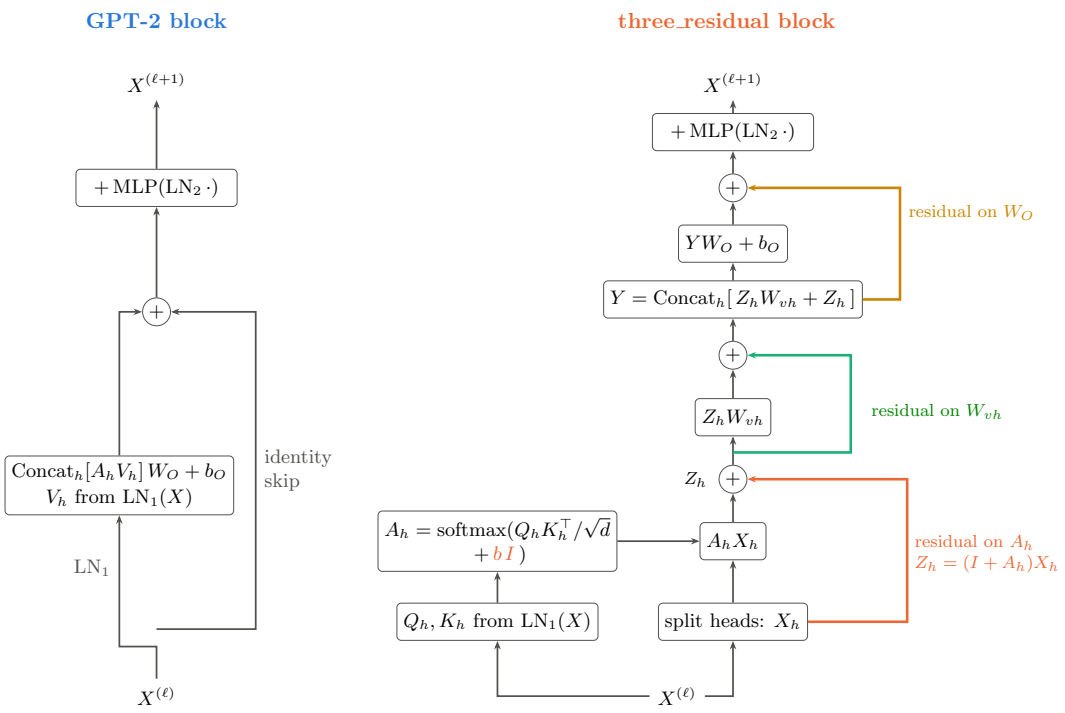


Figure 2: Left: the GPT-2 attention sublayer, an attention term plus an identity skip. Right: three_residual. There is no skip around the whole sublayer; instead each of the three matrices gets its own residual (colored arrows). The bias $b\,I$ on the self logit (orange) is used only during pretraining and is 0 in the final model.

2.2 What the three residuals do at initialization

Ignoring LayerNorm and biases, one head's output is $(I + A_h)\,X_h\,(I + W_{vh})$, followed by $(I + W_O)$. W_{vh} and W_O start small (standard deviation 0.02 and $0.02/\sqrt{24}$), so at initialization the block computes, per head,

$$X'_h \;\approx\; (I + A_h)\,X_h = X_h + A_hX_h.$$

The token's own state X_h is still there, but the averaged term A_hX_h enters at the *same* scale. That is the difference from the variants that trained in the earlier reports: in approach_3 the attention term is multiplied by a small W_O, so the stream starts out as almost pure X. The residuals on W_{vh} and W_O cannot help here. They act within each token's vector, while the averaging happens across tokens.

At initialization the attention is close to uniform (Figure 4, top row), so A_hX_h is roughly the mean of the earlier tokens. Each layer then replaces half of every token's state with that common mean. After 12 layers, the part of the stream that has never been averaged is about 2^{-12} of the whole, so all positions look nearly alike. That is the rank collapse measured in Section 3.

3 Initialization

3.1 The collapse probe

For a batch of 8 validation sequences, the probe runs the freshly initialized model and records, after every layer, the *distance from rank 1* of the residual stream: $\|X - \bar{X}\|/\|X\|$, where $\bar{X}$ repeats the

mean over positions (first 64 positions skipped). The value is 0 when every position holds the same vector. In the 2026-10-16 report, a layer-11 value of about 0.15 or less predicted collapse in every case. The plain three_residual sits at 0.148.

3.2 Two fixes: a diagonal bias and mimetic initialization

Diagonal bias. Add b to each head's self logit, $\mathrm{softmax}(QK^\top/\sqrt{d} + b\,I)$. At initialization the logits are close to 0, so query i gives itself weight about $e^b/(e^b + i - 1)$. To dominate around 1000 earlier positions, e^b must be well above 1000, so $b \gtrsim 7$. At $b = 8$ the measured self-weight at initialization is 0.84; at $b = 3$ and $b = 5$ it is only 0.05 and 0.26, which is not enough (Figure 3a).

Mimetic initialization (after Trockman & Kolter, 2023). Set $W_Q = W_K = s \cdot R$, with R a random orthogonal matrix. Every query then matches its own key best, with no extra parameter. Of the values tried, only $s = 1.5$ passes (Figure 3b).

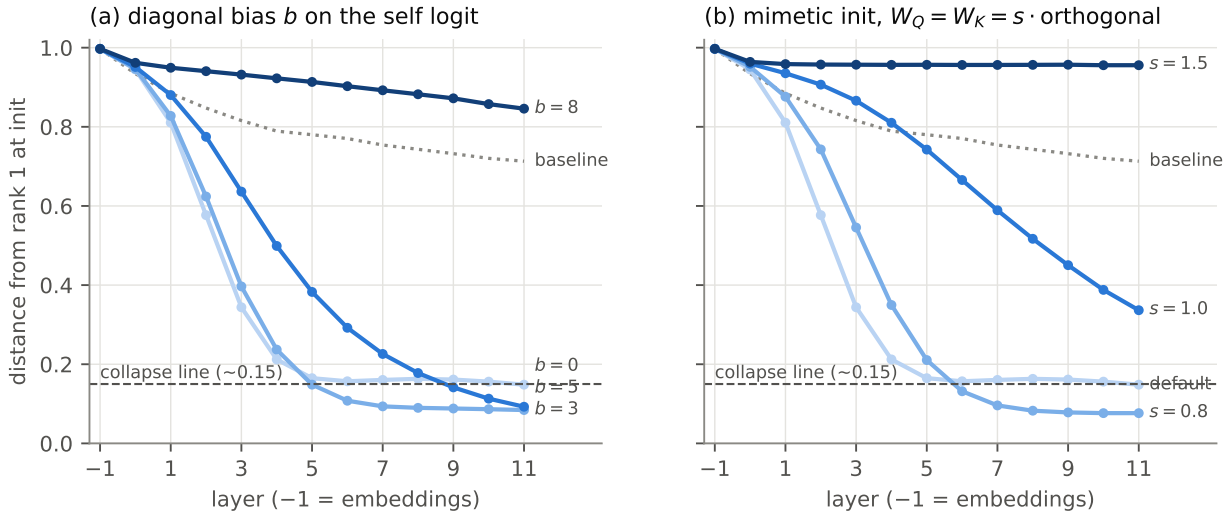


Figure 3: Distance from rank 1 of the residual stream after each layer, at initialization (8 validation sequences). The dotted gray line is standard GPT-2 at initialization. Plain three_residual ($b = 0$) falls below the collapse line by layer 5. Only $b = 8$ (layer 11: 0.846) and $s = 1.5$ (0.956) stay healthy. Curiously, $b = 3$ and $b = 5$ end even lower than $b = 0$.

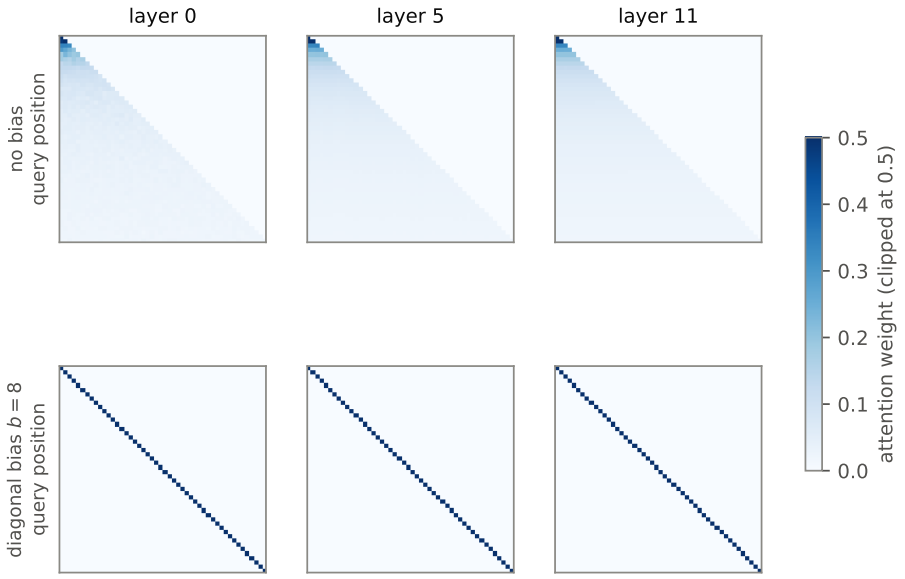


Figure 4: Attention at initialization, averaged over the 12 heads, for the first 48 tokens of one validation sequence. Without the bias every layer averages uniformly over the prefix (the triangle fades as rows get longer). With $b = 8$ each token attends almost only to itself, so $(I + A_h)X_h \approx 2X_h$ and nothing is averaged away.

3.3 Short runs: which fix trains

Each setting that passed the probe was trained for 250 steps (131M tokens), with the same schedule as a full run:

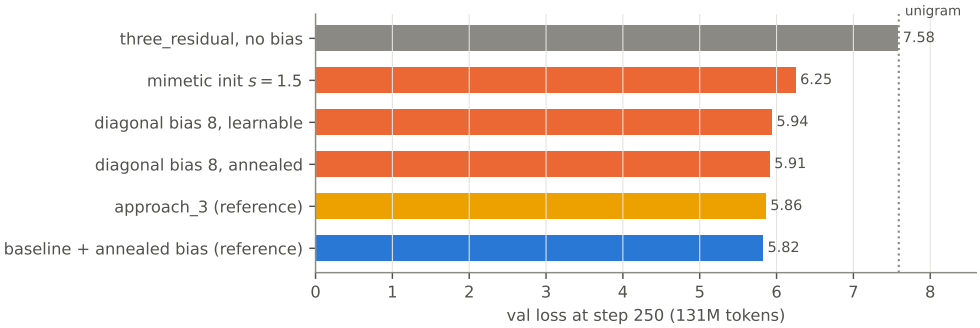


Figure 5: Validation loss at step 250. All three fixes escape the collapse. The two diagonal-bias settings are close to the healthy references; mimetic initialization lags, probably because its attention starts almost entirely on the token itself (self-weight 0.99) and takes longer to spread out.

The *annealed* bias was chosen for the full run. The learnable bias trains just as well, but it would leave an extra parameter in the final model. With annealing, the final model is exactly the three_residual block.

4 Pretraining

4.1 Bias schedule

The bias is a non-learned buffer, set before every optimizer step t to

$$b(t) = 8 \max\bigl(0,\ 1 - t/1000\bigr).$$

It reaches exactly 0 at step 1000 and stays there, so the last 3768 steps (79% of training) train the plain three_residual block. Everything else (data, order, seed, optimizer, learning rate) is identical to the other runs.

As a control, standard GPT-2 was trained with the same schedule (*baseline + annealed bias*). Comparing the two separates the cost of the block from any cost of the bias schedule. A plain GPT-2 baseline on OpenWebText has not been run yet.

4.2 Training curves

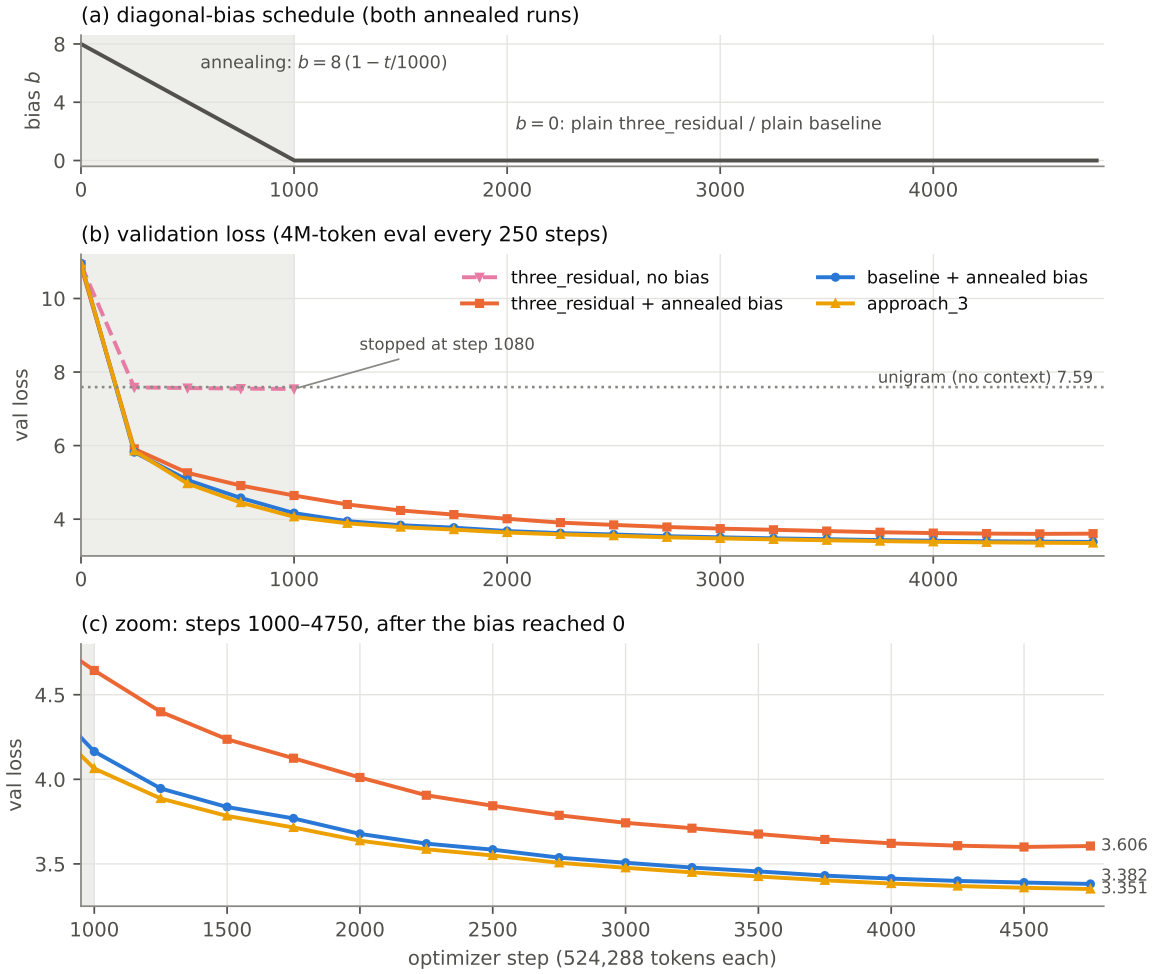


Figure 6: (a) Bias schedule; the shaded band marks steps 0–1000, while the bias is non-zero. (b) Validation loss. Plain three_residual stops at the unigram level within the first 250 steps; it was stopped at step 1080. With the annealed bias, three_residual follows the healthy runs during annealing and keeps improving after the bias reaches 0. (c) Zoom on steps 1000–4750: the gap to the control narrows from 0.48 nats at step 1000 to about 0.22.

- **No relapse when the bias reaches 0.** Validation loss is 4.64 at step 1000 (bias 0) and 4.40 at step 1250.
- **The gap to the control shrinks but does not close.** It falls from 0.48 nats at step 1000 to about 0.22 at the end, and it has hardly changed over the last 1000 steps.
- **Throughput.** The bias needs an additive attention mask, which rules out PyTorch's fastest attention kernel, so the run trained at 221k tokens/s against 332k for approach_3 (3.1 h instead of 2 h). The mask is kept even after the bias reaches 0. Switching back to the fast kernel at step 1000 would recover most of that time.

4.3 Final results

Model	Parameters	Final val loss	Note
three_residual, no bias	117,978,624	—	collapsed; 7.54 at step 1000, stopped
three_residual + annealed bias	117,978,624	3.620	bias 0 from step 1000
baseline + annealed bias	124,475,904	3.405	control
approach_3	124,475,904	3.376	from the 2026-10-19 OpenWebText run
unigram	—	7.591	token frequencies only

three_residual ends 0.215 nats behind the control and 0.244 behind approach_3.

5 Attention

All heatmaps show the first 48 tokens of one validation sequence. All numbers (self-weight, distance from rank 1) are averages over 8 validation sequences and query positions ≥ 64.

5.1 Collapsed vs. trained

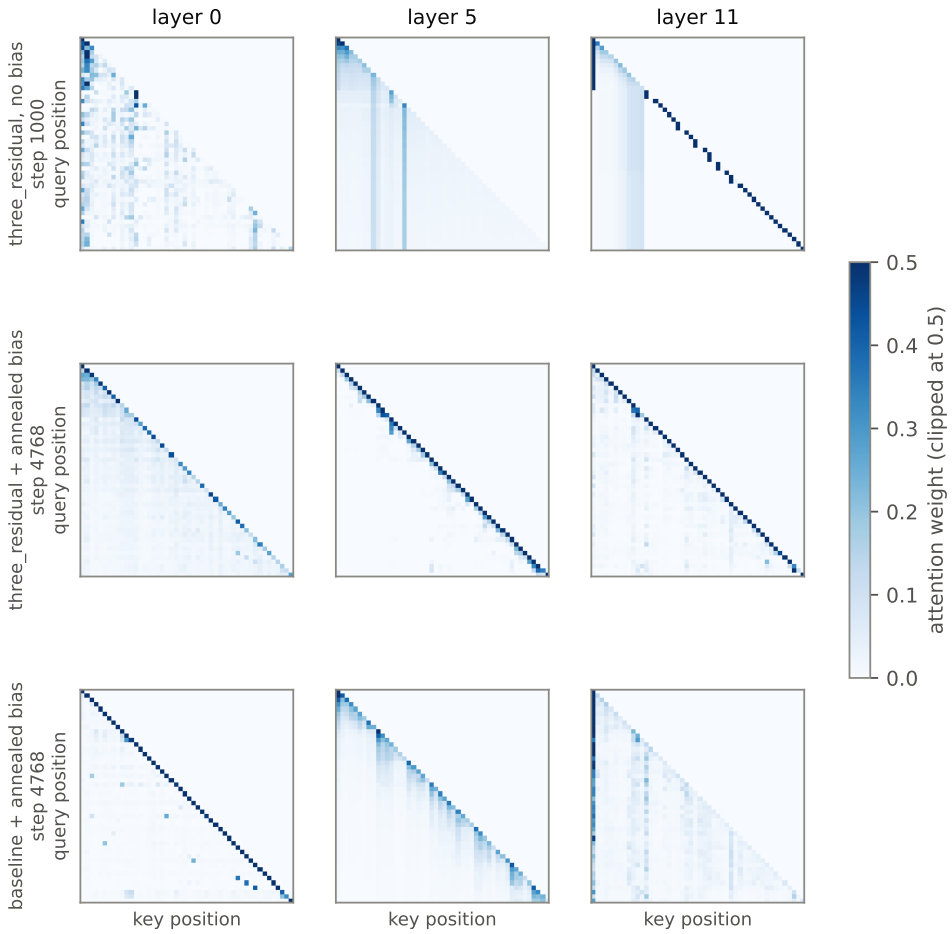


Figure 7: Attention averaged over heads, at layers 0, 5 and 11. Top: plain three_residual at step 1000, after collapsing. Middle: three_residual trained with the annealed bias (step 4768, bias 0). Bottom: the GPT-2 control trained with the same schedule.

- **Collapsed three_residual (top).** Over 8 sequences its self-weight is low (at most 0.16 per layer), and many heads read the same block of early positions. Every token gets the same output, and the residual stream ends almost rank 1 (layer 11: 0.060, Figure 10). In this one sequence, a few heads in layer 11 look sharply diagonal in the first 48 tokens; averaged over 8 sequences that pattern is weak (the token itself is the top key in only 15–30% of cases). An earlier quick look, based on this single sequence, overstated it.
- **Trained three_residual (middle).** Strongly diagonal at every depth: each token attends mostly to itself, with a little extra weight on nearby earlier tokens.
- **Control (bottom).** Layer 0 is diagonal, a leftover of the bias. Deeper layers show the usual GPT-2 patterns: attention to the previous few tokens, stripes on important earlier tokens, and a column on the first token.

5.2 Every head of the trained three_residual

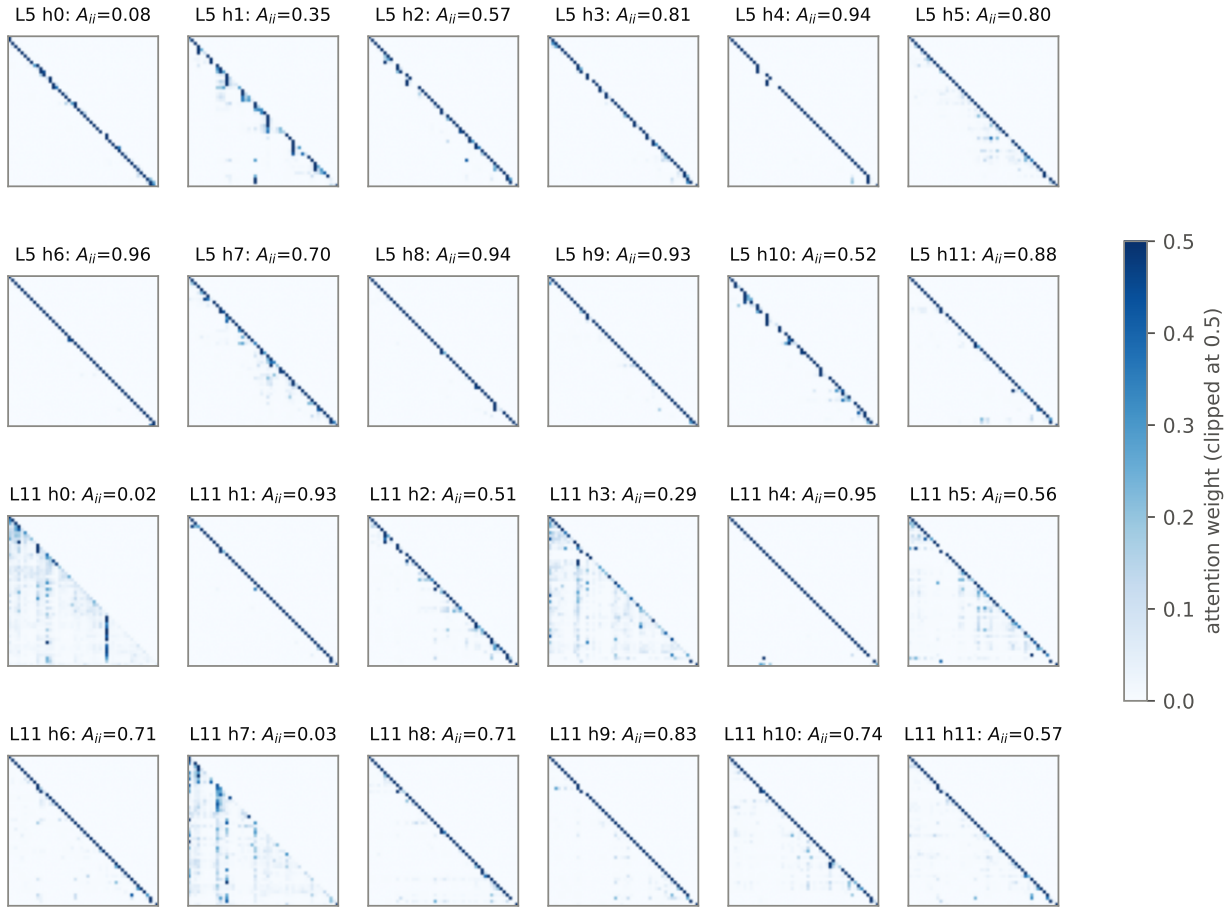


Figure 8: All 12 heads of the trained three_residual at layers 5 and 11, with each head's mean self-weight A_{ii}. Most heads are almost pure self-attention. A few mix across tokens: L11 h0, h3 and h7 spread their weight over earlier tokens, and L5 h1 and h10 attend to recent tokens. Some heads that look diagonal attend to the *previous* token, one step below the diagonal: L5 h0 looks diagonal but has $A_{ii} = 0.08$.

5.3 How much each head attends to itself

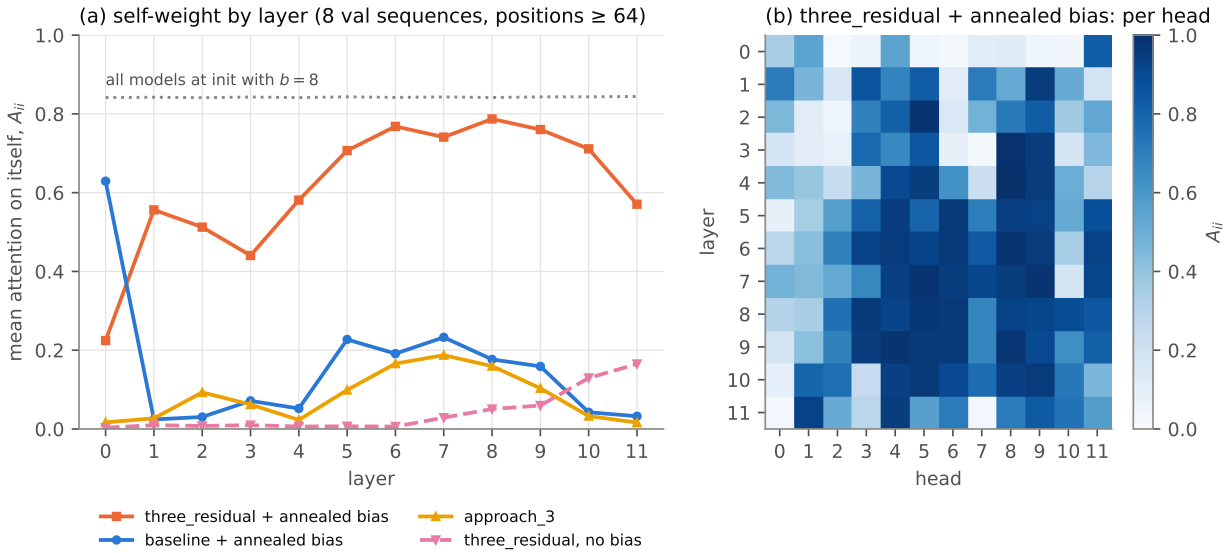


Figure 9: (a) Mean self-weight A_{ii} per layer. All runs with the bias start at 0.84 (dotted line). After training, the control drops below 0.25 everywhere except layer 0, while three_residual stays between 0.44 and 0.79 in layers 1–11. (b) Self-weight of every head in the trained three_residual.

	mean A_{ii}	heads with $A_{ii} > 0.5$	weight on the 8 previous tokens
three_residual + annealed bias	0.61	94 / 144	0.21 (highest in layers 1–5)
baseline + annealed bias	0.16	14 / 144	0.28 (highest in layers 5–8)
approach_3	0.08	1 / 144	0.23 (highest in layers 5–8)

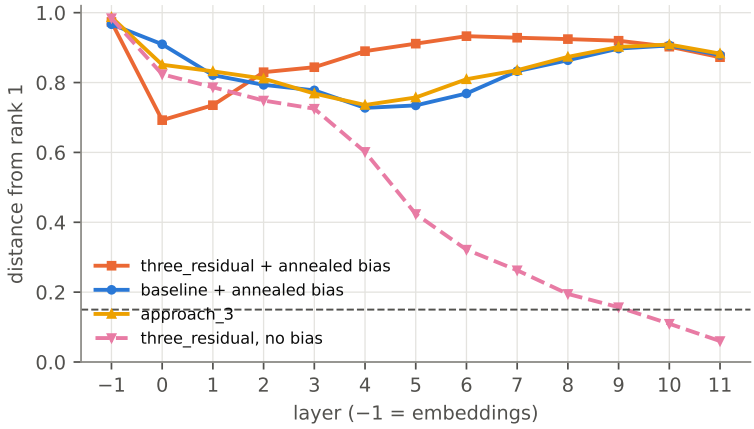


Figure 10: Distance from rank 1 of the residual stream in the trained models. The trained three_residual keeps the positions distinct (layer 11: 0.873), as healthy as the control and approach_3. The collapsed model falls to 0.060.

5.4 Interpretation

The control drops its diagonal as soon as the bias is gone, because GPT-2's identity skip already carries each token's state. three_residual has no small-scale path of that kind: the averaged term $A_h X_h$ always enters at the same scale as X_h (Equation 2), and W_{vh} and W_O only act within each token. The only way it can keep a token's own state dominant is to make A_h close to I, and that is what most of its heads learn.

Two consequences:

- **Interpretability.** In three_residual, head h moves information between tokens only through $Z_h = (I + A_h)X_h$, and the identity and the attention enter at a fixed, equal scale. So the row-normalized matrix $\frac{1}{2}(I + A_h)$ is head h's exact token-mixing matrix on the X path: it says how much of token i's new state is its own, and how much comes from each earlier token. In approach_3 or GPT-2 this cannot be read off A_h, because the attention term is rescaled by learned weights before it meets the skip.
- **A likely cost.** Heads that are busy copying the token itself do little across tokens. In three_residual, only 50 of 144 heads put less than half their weight on the token itself, against 130 in the control. This is a plausible reason for the 0.215-nat gap, but it has not been tested.

6 Limitations

- One seed per configuration; the seed-to-seed spread on OpenWebText has not been measured (on FineWeb-Edu it was assumed to be about ± 0.01).
- No plain GPT-2 baseline on OpenWebText yet; the control uses the same bias schedule.
- Mimetic initialization and the learnable bias were trained for only 250 steps.
- Heatmaps come from one sequence; the numbers in the tables are averaged over 8 sequences.
- The collapse line (about 0.15) is an empirical rule from the earlier variants, not a derived threshold.

7 Next steps

1. Train a plain GPT-2 baseline on OpenWebText, for a clean reference.
2. Test the head-capacity explanation: for example, scale the self term, $Z_h = A_h X_h + \gamma X_h$ with a learned γ, so heads can keep the token's state without spending their attention on it.
3. Run mimetic initialization ($s = 1.5$) for the full 2.5B tokens. It needs no schedule and no extra parameter.
4. Switch to the fast attention kernel once the bias reaches 0 (about $1.5\times$ faster).

Files

`code/gpt2_torch.py`	PyTorch model used for training; variant `three_residual`, options `diag_bias`, `diag_learn`, `mimetic`
`code/model.py`	TensorFlow `src/model.py` with `three_residual=True`
`code/three_residual_analysis.py`	probe, self-weight and attention maps → `data/`
`code/make_three_residual_report.py`	figures and `numbers.tex`
`data/`	`probe_init.json`, `trained.json`, `attention.npz`, `summary.json`
Runs	`/local/scratch/a/liu3170/runs/openwebtext/`: `tr_diag8_anneal1000`, `baseline_diag8_anneal1000`, `three_residual` (collapsed), `tr_diag8_learn_s250`, `tr_mimetic1.5_s250`, `approach_3`
Command	`python train.py --variant three_residual --diag_bias 8 --anneal_steps 1000 --data <openwebtext> --out <dir>`

A Code: the attention forward pass

```
def attend(self, q, k, v):
    if hasattr(self, 'diag'):  # diagonal bias b on the self logit (training aid)
        T = q.shape[-2]
        bias = self.causal_neg[:T, :T] + self.diag.view(-1, 1, 1) * self.eye[:T, :T]  # [H, T, T]
        return F.scaled_dot_product_attention(q, k, v, attn_mask=bias.to(q.dtype))
    return F.scaled_dot_product_attention(q, k, v, is_causal=True)

# three_residual branch of Attention.forward(x_ln, x_raw)
q, k = map(self.split_heads, self.c_attn(x_ln).split(D, dim=2))  # Q, K from LN_1(X)
xh = self.split_heads(x_raw)                                     # X_h: raw residual stream
z = self.attend(q, k, xh) + xh                                   # Z_h = A_h X_h + X_h
z = torch.einsum('bhtd,hde->bhte', z, self.w_v) + z              # Z_h W_vh + Z_h
y = z.transpose(1, 2).contiguous().view(B, T, D)                 # Y = Concat_h
return self.c_proj(y) + y                                        # Y W_O + b_O + Y

# train.py, before every optimizer step
if args.anneal_steps:
    model.set_diag(args.diag_bias * max(0.0, 1 - step / args.anneal_steps))
```